

Chapter 6

Statistical Models for Network Graphs

6.1 Introduction

The network models discussed in the previous chapter serve a variety of useful purposes. Yet for the purpose of statistical model building, they come up short. Indeed, as Robins and Morris [125] write, “A good [statistical network graph] model needs to be both estimable from data and a reasonable representation of that data, to be theoretically plausible about the type of effects that might have produced the network, and to be amenable to examining which competing effects might be the best explanation of the data.” None of the models we have seen up until this point are really intended to meet such criteria.

In contrast, there are a number of other classes of network graph models which *are* designed explicitly for use as statistical models. In fact, the three main such classes of models developed to date closely parallel more familiar statistical models for non-network datasets. The class of exponential random graph models are analogous to standard regression models—particularly, generalized linear models. Similarly, stochastic block models draw their inspiration from mixture models, as they are, in their most basic form, essentially a mixture of classical random graph models. Finally, latent network models are a network-based variant of the common practice of using both observed and unobserved (i.e., latent) variables in modeling an outcome (i.e., in this case, the presence or absence of network edges).

It is important to note, however, that none of these models are simply direct implementations of their classical analogues. The adaptation of the latter to network-based data structures can have nontrivial implications on model specification and identifiability, model fitting, and the assessment of significance of terms in the model and model goodness of fit.

In this chapter we explore the basic structure and use of certain canonical examples of each of these three classes of statistical models for network graphs.

6.2 Exponential Random Graph Models

Exponential random graph models (ERGMs)¹ are designed in direct analogy to the classical generalized linear models (GLMs). They are formulated in a manner that is intended to facilitate the adaptation and extension of well-established statistical principles and methods for the construction, fitting, and comparison of models. Nevertheless, the appropriate specification and fitting of ERGMs can be decidedly more subtle than with standard GLMs. Moreover, much of the standard inferential infrastructure available for GLMs, resting on asymptotic approximations to appropriate chi-square distributions, has yet to be formally justified in the case of ERGMs. As a result, while this class of models arguably has substantial potential, in practice it must be used with some care.

6.2.1 General Formulation

Consider $G = (V, E)$ as a random graph. Let $Y_{ij} = Y_{ji}$ be a binary random variable indicating the presence or absence of an edge $e \in E$ between the two vertices i and j in V . The matrix $\mathbf{Y} = [Y_{ij}]$ is thus the (random) adjacency matrix for G . Denote by $\mathbf{y} = [y_{ij}]$ a particular realization of $\mathbf{Y}$. An exponential random graph model is a model specified in exponential family form² for the joint distribution of the elements in $\mathbf{Y}$. The basic specification for an ERGM is a model of the form

$$\mathbb{P}_{\theta}(\mathbf{Y} = \mathbf{y}) = \left(\frac{1}{\kappa}\right) \exp \left\{ \sum_H \theta_H g_H(\mathbf{y}) \right\}, \quad (6.2)$$

where

- (i) each H is a *configuration*, which is defined to be a set of possible edges among a subset of the vertices in G ;
- (ii) $g_H(\mathbf{y}) = \prod_{y_{ij} \in H} y_{ij}$, and is therefore either one if the configuration H occurs in $\mathbf{y}$, or zero, otherwise;

¹ These models have also been referred to as p^* models, particularly in the social network literature, where they are seen as one of the later examples of a series of model classes introduced in succession over a roughly 20-year period covering the late 1970s, 1980s, and early 1990s. See the review of Wasserman and Pattison [145], for example. Our use of the term ‘exponential random graph models’ reflects current practice, which emphasizes the connection of these models with traditional exponential family models in classical statistics.

² Recall that an arbitrary (discrete) random vector $\mathbf{Z}$ is said to belong to an *exponential family* if its probability mass function may be expressed in the form

$$\mathbb{P}_{\theta}(\mathbf{Z} = \mathbf{z}) = \exp \{ \theta^T \mathbf{g}(\mathbf{z}) - \psi(\theta) \}, \quad (6.1)$$

where $\theta \in \mathbb{R}^p$ is a $p \times 1$ vector of parameters, $\mathbf{g}(\cdot)$ is a p -dimensional function of $\mathbf{z}$, and $\psi(\theta)$ is a normalization term, ensuring that $\mathbb{P}_{\theta}(\cdot)$ sums to one.

- (iii) a non-zero value for θ_H means that the Y_{ij} are dependent for all pairs of vertices $\{i, j\}$ in H , conditional upon the rest of the graph; and
- (iv) $\kappa = \kappa(\theta)$ is a normalization constant,

$$\kappa(\theta) = \sum_{\mathbf{y}} \exp \left\{ \sum_H \theta_H g_H(\mathbf{y}) \right\}. \quad (6.3)$$

Note that the summation in (6.2) is over all possible configurations H . Importantly, given a choice of functions g_H and their coefficients θ_H , this implies a certain (in)dependency structure among the elements in $\mathbf{Y}$, which is, of course, appealing, given the inherently relational nature of a network. Generally speaking, such structure typically can be described as specifying that the random variables $\{Y_{ij}\}_{(i,j) \in \mathcal{A}}$ are independent of $\{Y_{i'j'}\}_{(i',j') \in \mathcal{B}}$, conditional on the values of $\{Y_{i''j''}\}_{(i'',j'') \in \mathcal{C}}$, for some given index sets $\mathcal{A}$, $\mathcal{B}$, and $\mathcal{C}$. Conversely, we can begin with a collection of (in)dependence relations among subsets of elements in $\mathbf{Y}$ and try to derive the induced form of the (g_H, θ_H) pairs.³

The ERGM framework allows for a number of variations and extensions. For example, directed versions of ERGMs are also available. In addition, in defining ERGMs for either undirected or directed graphs, it is straightforward to include, if desired, information on vertices beyond their connectivity, such as actor attributes in a social network or known functionalities of proteins in a network of protein interactions. Given a realization $\mathbf{x}$ of a random vector $\mathbf{X}$ on the vertices in G , we simply specify an exponential form for the conditional distribution $\mathbb{P}_\theta(\mathbf{Y} = \mathbf{y} | \mathbf{X} = \mathbf{x})$ that involves additional statistics $g(\cdot)$ that are functions of both $\mathbf{y}$ and $\mathbf{x}$.

In this section, we will illustrate the construction, fitting, and assessment of ERGMs using the *lazega* data set on collaboration among lawyers, introduced in Chap. 1. Within R, easily the most comprehensive and sophisticated package for ERGMs is the **ergm** package, which is part of the **statnet** suite of packages.⁴ Since **ergm** uses the **network** package to represent network objects, we convert the **igraph** object *lazega* to the format used in **statnet**, first separating the network into adjacency matrix and attributes

```
#6.1 1 > library(sand)
      2 > data(lazega)
      3 > A <- get.adjacency(lazega)
      4 > v.attrs <- get.data.frame(lazega, what="vertices")
```

and then creating the analogous network object for **ergm**

```
#6.2 1 > library(ergm) # Will load package 'network' as well.
      2 > lazega.s <- network::as.network(as.matrix(A),
```

³ However, it is important to realize that it is *not* the case that simply any collection of (in)dependence relations among the elements of $\mathbf{Y}$ yields a proper joint distribution on $\mathbf{Y}$. Rather, certain conditions must be satisfied, as formalized in the celebrated Hammersley-Clifford theorem (e.g., Besag [12]).

⁴ The **statnet** suite is arguably the most sophisticated single collection of R packages for doing statistical modeling of network graphs, particularly from the perspective of social network analysis.

```

3 + directed=FALSE)
4 > network::set.vertex.attribute(lazega.s, "Office",
5 +   v.attrs$Office)
6 > network::set.vertex.attribute(lazega.s, "Practice",
7 +   v.attrs$Practice)
8 > network::set.vertex.attribute(lazega.s, "Gender",
9 +   v.attrs$Gender)
10 > network::set.vertex.attribute(lazega.s, "Seniority",
11 +   v.attrs$Seniority)

```

6.2.2 Specifying a Model

The general formulation just described leaves much flexibility in specifying an ERGM. We illustrate in the material that follows below, but refer the reader to, for example, the review article by Robins et al. [126] or the book by Lusher et al. [105] for a more comprehensive treatment.

We have already seen an example of what is arguably the simplest ERGM, in the form of the Bernoulli random graph model Chap. 5.2. To see this, suppose we specify that, for a given pair of vertices, the presence or absence of an edge between that pair is independent of the status of possible edges between any other pairs of vertices.⁵ Then $\theta_H = 0$ for all configurations H involving three or more vertices. As a result, the ERGM in (6.2) reduces to

$$\mathbb{P}_{\theta}(\mathbf{Y} = \mathbf{y}) = \left(\frac{1}{\kappa}\right) \exp \left\{ \sum_{i,j} \theta_{ij} y_{ij} \right\}. \quad (6.4)$$

Furthermore, if we assume that the coefficients θ_{ij} are equal to some common value θ (typically referred to as an assumption of *homogeneity* across the network), then (6.4) further simplifies to

$$\mathbb{P}_{\theta}(\mathbf{Y} = \mathbf{y}) = \left(\frac{1}{\kappa}\right) \exp \{ \theta L(\mathbf{y}) \}, \quad (6.5)$$

where $L(\mathbf{y}) = \sum_{i,j} y_{ij} = N_e$ is the number of edges in the graph. The result is equivalent to a Bernoulli random graph model, with $p = \exp(\theta) / [1 + \exp(\theta)]$.

To specify models in **ergm**, we use the function `formula` and standard **R** syntax. For example, model (6.5) may be specified for the network `lazega.s` as

```

#6.3 1 > my.ergm.bern <- formula(lazega.s ~ edges)
      2 > my.ergm.bern
      3 lazega.s ~ edges

```

in which case the statistic L takes the value

⁵ That is, for each pair $\{i, j\}$, we assume that Y_{ij} is independent of $Y_{i',j'}$, for any $\{i', j'\} \neq \{i, j\}$.

```
#6.4 1 > summary.statistics(my.ergm.bern)
      2 edges
      3   115
```

The strength of ERGMs lies in our ability to specify decidedly more nuanced models than that above. Doing so properly and effectively, however, requires some thought and care.

To begin, note that the model in (6.5) can be thought of as specifying that the log-odds of observing a given network G (or, more specifically, its adjacency matrix $\mathbf{y}$) is simply proportional to the number of edges in the network—arguably the most basic of network statistics. Traditionally, it has been of interest to also incorporate analogous statistics of higher-order global network structure, such as counts of k -stars,⁶ say $S_k(\mathbf{y})$, and of triangles, say $T(\mathbf{y})$. Frank and Strauss [58] show that models of the form

$$\mathbb{P}_{\theta}(\mathbf{Y} = \mathbf{y}) = \left(\frac{1}{\kappa}\right) \exp \left\{ \sum_{k=1}^{N_v-1} \theta_k S_k(\mathbf{y}) + \theta_T T(\mathbf{y}) \right\} \quad (6.6)$$

are equivalent to a certain limited form of dependence among the edges y_{ij} , in contrast to the independence specified by the Bernoulli model.⁷

In using such models, common practice has been to include star counts S_k no higher than $k = 2$, or at most $k = 3$, by setting $\theta_4 = \dots = \theta_{N_v-1} = 0$. For example,

```
#6.5 1 > my.ergm <- formula(lazega.s ~ edges + kstar(2)
      2 +   + kstar(3) + triangle)
      3 > summary.statistics(my.ergm)
      4   edges   kstar2   kstar3 triangle
      5     115     926    2681     120
```

While simpler and, ideally, more interpretable, than the general formulation in (6.6), experience nevertheless has shown this practice to frequently produce models that fit quite poorly to real data. Investigation of this phenomena has found it to be intimately related to the issue of model degeneracy.⁸ See Handcock [68]. Unfortunately, the alternative—including a sufficiently large number of higher order terms—is problematic as well, from the perspective of model fitting.

A solution to this dilemma, proposed by Snijders et al. [134], is to impose a parametric constraint of the form $\theta_k \propto (-1)^k \lambda^{2-k}$ upon the star parameters, for all $k \geq 2$, for some $\lambda \geq 1$. This tactic has the effect of combining all of the k -star statistics $S_k(\mathbf{y})$ in (6.6), for $k \geq 2$, into a single *alternating k -star statistic* of the form

$$AKS_{\lambda}(\mathbf{y}) = \sum_{k=2}^{N_v-1} (-1)^k \frac{S_k(\mathbf{y})}{\lambda^{k-2}}, \quad (6.7)$$

⁶ Note that $S_1(\mathbf{y}) = N_e$ is the number of edges.

⁷ Formally, Frank and Strauss introduced the notion of *Markov dependence* for network graph models, which specifies that two possible edges are dependent whenever they share a vertex, conditional on all other possible edges. A random graph G arising under Markov dependence conditions is called a *Markov graph*.

⁸ In this context the term is used to refer to a probability distribution that places a disproportionately large amount of its mass on a correspondingly small set of outcomes.

and weighting that statistic by a single parameter θ_{AKS} that takes into account the star effects of all orders simultaneously. One may think of the alternating signs in (6.7) as allowing the counts of k -stars of successively greater order to balance each other, rather than simply ballooning (i.e., since more k -stars, for a given k , means more k' -stars, for $k' < k$).

Alternatively, and equivalently if the number of edges is included in the model, there is the *geometrically weighted degree count*, defined as

$$GWD_{\gamma}(\mathbf{y}) = \sum_{d=0}^{N_v-1} e^{-\gamma d} N_d(\mathbf{y}) , \quad (6.8)$$

where $N_d(\mathbf{y})$ is the number of vertices of degree d and $\gamma > 0$ is related to λ through the expression $\gamma = \log[\lambda / (\lambda - 1)]$. This approach in a sense attempts to model the degree distribution, with choice of γ influencing the extent to which higher-degree vertices are likely to occur in the graph G .

Snijders et al. [134] discuss a number of other similar statistics, including a generalization of triadic structures based on alternating sums of k -triangles, which takes the form⁹

$$AKT_{\lambda}(\mathbf{y}) = 3T_1 + \sum_{k=2}^{N_v-2} (-1)^{k+1} \frac{T_k(\mathbf{y})}{\lambda^{k-1}} . \quad (6.9)$$

Here T_k is the number of k -triangles, where a k -triangle is defined to be a set of k individual triangles sharing a common base. A discussion of the type of dependency properties induced among edges y_{ij} by such statistics can be found in Pattison and Robins [122].

These three statistics can be used in **ergm** by specifying terms `altkstar`, `gwdegree`, or `gwesp`, respectively, in the model. For example,

```
#6.6 1 > my.ergm <- formula(lazega.s ~ edges
      2 + + gwesp(1, fixed=TRUE))
      3 > summary.statistics(my.ergm)
      4 edges gwesp.fixed.1
      5 115.0000 213.1753
```

Note that all of the model specifications discussed so far involve statistics that are functions only of the network $\mathbf{y}$ (i.e., controlling for endogenous effects). Yet it is natural to expect that the chance of an edge joining two vertices depends not only on the status (i.e., presence or absence) of edges between other vertex pairs, but also on attributes of the vertices themselves (i.e., allowing for assessment of exogenous effects). For attributes that have been measured, we can incorporate them into the types of ERGMs we have seen, in the form of additional statistics in the exponential term in (6.2), with the normalization constant κ modified analogously, according to (6.3).

⁹ Hunter [78] offers an equivalent formulation of this definition, in terms of geometrically weighted counts of the neighbors common to adjacent vertices.

One natural form for such statistics is

$$g(\mathbf{y}, \mathbf{x}) = \sum_{1 \leq i < j \leq N_v} y_{ij} h(\mathbf{x}_i, \mathbf{x}_j), \quad (6.10)$$

where h is a symmetric function of $\mathbf{x}_i$ and $\mathbf{x}_j$, and $\mathbf{x}_i$ (or $\mathbf{x}_j$) is the vector of observed attributes for the i th (or j th) vertex. Intuitively, if h is some measure of ‘similarity’ in attributes, then the statistic in (6.10) assess the total similarity among network neighbors.

Two common choices of h produce analogues of ‘main effects’ and ‘second-order effects’ (or similarity or homophily effects) of certain attributes. Main effects, for a particular attribute x , are defined using a simple additive form:

$$h(x_i, x_j) = x_i + x_j. \quad (6.11)$$

On the other hand, second-order effects are defined using an indicator for equivalence of the respective attribute between two vertices, i.e.,

$$h(x_i, x_j) = I\{x_i = x_j\}. \quad (6.12)$$

Main effects and second-order effects may be incorporated into a model within **ergm** using the terms `nodemain` and `nodematch`, respectively.

To summarize, the various statistics introduced above have been chosen only to illustrate the many types of effects that may be captured in modeling network graphs using ERGMs. In modeling the network `lazega.s` throughout the rest of this section, we will draw on the analyses of Hunter and Handcock [79] and Snijders et al. [134]. In particular, we will specify a model of the form

$$\mathbb{P}_{\theta, \beta}(\mathbf{Y} = \mathbf{y} | \mathbf{X} = \mathbf{x}) = \left(\frac{1}{\kappa(\theta, \beta)} \right) \exp \{ \theta_1 S_1(\mathbf{y}) + \theta_2 AKT_\lambda(\mathbf{y}) + \beta^T \mathbf{g}(\mathbf{y}, \mathbf{x}) \}, \quad (6.13)$$

where $\mathbf{g}$ is a vector of five attribute statistics and β is the corresponding vector of parameters.

In **R**, our model is expressed as

```
#6.7 1 > lazega.ergm <- formula(lazega.s ~ edges
2 +   + gwspe(log(3), fixed=TRUE)
3 +   + nodemain("Seniority")
4 +   + nodemain("Practice")
5 +   + match("Practice")
6 +   + match("Gender")
7 +   + match("Office"))
```

This specification allows us to control for the density of the network and some effects of transitivity. In addition, it allows us to assess the effect on the formation of collaborative ties among lawyers that is had by seniority, the type of practice (i.e., corporate or litigation), and commonality of practice, gender, and office location.

6.2.3 Model Fitting

In standard settings, with independent and identically distributed realizations, exponential family models like that in (6.1) are generally fit using the method of maximum likelihood. In the context of the ERGMs in (6.2), the maximum likelihood estimators (MLEs) $\hat{\theta}_H$ of the parameters θ_H are well defined, assuming an appropriately-specified model, but their calculation is non-trivial.

Consider the general definition of an ERGM in (6.2). The MLE for the vector $\theta = (\theta_H)$ is defined as $\hat{\theta} = \arg \max_{\theta} \ell(\theta)$, where $\ell(\theta)$ is the log-likelihood, which has the particularly simple form common to exponential families,

$$\ell(\theta) = \theta^T \mathbf{g}(\mathbf{y}) - \psi(\theta) . \quad (6.14)$$

Here $\mathbf{g}$ denotes the vector of functions g_H and $\psi(\theta) = \log \kappa(\theta)$. Alternatively, taking derivatives on each side and using the fact that $\mathbb{E}_{\theta}[\mathbf{g}(\mathbf{Y})] = \partial \psi(\theta) / \partial \theta$, the MLE can also be expressed as the solution to the system of equations

$$\mathbb{E}_{\hat{\theta}}[\mathbf{g}(\mathbf{Y})] = \mathbf{g}(\mathbf{y}) . \quad (6.15)$$

Unfortunately, the function $\psi(\theta)$, occurring in both (6.14) and (6.15), cannot be evaluated explicitly in any but the most trivial of settings, as it involves the summation in (6.3) over $2^{\binom{N_Y}{2}}$ possible choices of $\mathbf{y}$, for each candidate θ . Therefore, it is necessary to use numerical methods to compute approximate values for $\hat{\theta}$.

In **ergm**, models are fit using the function `ergm`, which implements a version of Markov chain Monte Carlo maximum likelihood estimation, deriving from the fundamental work of Geyer and Thompson [62]. See Hunter and Handcock [79], for example, for additional details and references. Our model in (6.13), for example, is fit as

```
#6.8 1 > set.seed(42)
      2 > lazega.ergm.fit <- ergm(lazega.ergm)
```

The analogy between ERGMs and GLMs may be drawn upon in summarizing and assessing the fit of the former.¹⁰ For example, examination of an analysis of variance (ANOVA) table indicates that there is strong evidence that the variables used in the model `lazega.ergm` explain the variation in network connectivity to a highly nontrivial extent, with a change in deviance of 459 with only seven variables.

```
#6.9 1 > anova.ergm(lazega.ergm.fit)
      2 Analysis of Variance Table
      3
      4 Model 1: lazega.s ~ edges + gwesp(log(3), fixed = TRUE) +
      5       nodemain("Seniority") + nodemain("Practice") +
      6       match("Practice") + match("Gender") +
```

¹⁰ We note that the **ergm** package provides not only summary statistics but also p -values. However, as mentioned earlier, the theoretical justification for the asymptotic chi-square and F -distributions used by **ergm** to compute these values has not been established formally to date. Therefore, our preference is to interpret these values informally, as additional summary statistics.


```

7      match("Office")
8      Df Deviance Resid. Df Resid. Dev Pr(>|Chisq|)
9 NULL                                630      0.00
10 Model 1:  7   -458.86      623    458.86    < 2.2e-16
11
12 NULL
13 Model 1: ***
14 ---
15 Signif. codes:  0 *** 0.001 ** 0.01 * 0.05 . 0.1 1

```

Similarly, we can examine the relative contribution of the individual variables in our model.

```

#6.10 1 > summary.ergm(lazega.ergm.fit)
2 =====
3 Summary of model fit
4 =====
5
6 Formula: lazega.s ~ edges + gwesp(log(3), fixed = TRUE) +
7      nodemain("Seniority") + nodemain("Practice") +
8      match("Practice") + match("Gender") +
9      match("Office")
10
11 Iterations: 20
12
13 Monte Carlo MLE Results:
14
15      Estimate Std. Error MCMC %
16 edges -6.98047    0.72739      0
17 gwesp.fixed.1.09861228866811 0.58967    0.08786      0
18 nodecov.Seniority 0.02442    0.00675      0
19 nodecov.Practice 0.39538    0.11013      0
20 nodematch.Practice 0.76438    0.20055      0
21 nodematch.Gender 0.72110    0.25167      0
22 nodematch.Office 1.16155    0.19498      0
23      p-value
24 edges < 1e-04 ***
25 gwesp.fixed.1.09861228866811 < 1e-04 ***
26 nodecov.Seniority 0.000321 ***
27 nodecov.Practice 0.000357 ***
28 nodematch.Practice 0.000152 ***
29 nodematch.Gender 0.004308 **
30 nodematch.Office < 1e-04 ***
31 ---
32 Signif. codes:  0 *** 0.001 ** 0.01 * 0.05 . 0.1 1
33
34 Null Deviance: 873.4 on 630 degrees of freedom
35 Residual Deviance: 458.9 on 623 degrees of freedom
36
37 AIC: 472.9 BIC: 504 (Smaller is better.)

```

In order to interpret the coefficients, it is useful to think in terms of the probability of a given vertex pair having an edge, conditional on the edge status between all

other pairs. Writing $\mathbf{Y}_{(-ij)}$ to be all of the elements of $\mathbf{Y}$ except Y_{ij} , the distribution of Y_{ij} conditional on $\mathbf{Y}_{(-ij)}$ is Bernoulli and satisfies the expression

$$\log \left[\frac{\mathbb{P}_{\theta}(Y_{ij} = 1 | \mathbf{Y}_{(-ij)} = \mathbf{y}_{(-ij)})}{\mathbb{P}_{\theta}(Y_{ij} = 0 | \mathbf{Y}_{(-ij)} = \mathbf{y}_{(-ij)})} \right] = \theta^T \Delta_{ij}(\mathbf{y}) , \quad (6.16)$$

where $\Delta_{ij}(\mathbf{y})$ is the *change statistic*, denoting the difference between $\mathbf{g}(\mathbf{y})$ when $y_{ij} = 1$ and when $y_{ij} = 0$,

So the estimated coefficient of each attribute statistic in this analysis may be interpreted as a conditional log-odds ratio for cooperation between lawyers. For example, practicing corporate law, rather than litigation, increases the odds of cooperation by a factor of $\exp(0.3954) \approx 1.485$, or nearly 50 %. Similarly, being of the same gender more than doubles the odds of cooperation, since $\exp(0.7211) \approx 2.057$. In all cases, such statements hold in the sense of ‘all else being equal’ (i.e., given no change among values of the other statistics). Note too that for all of the variables but one the coefficient differs from zero by at least one standard error, suggesting some nontrivial effect of these variables on the formation of network ties.

Similarly, in terms of network structure, the magnitude of the coefficient $\hat{\theta}_2 \approx 0.5897$ for the alternating k -triangle statistic and the comparatively small corresponding standard error indicate that there is also evidence for a nontrivial transitivity effect. Note that, given the inclusion of our second-order attribute statistics in the model, our quantification of this effect naturally controls for basic homophily on these attributes. So there is likely something other than similarity of gender, practice, and office at work here—possibly additional attributes we have not controlled for, or possibly social processes of team formation.

6.2.4 Goodness-of-Fit

In any sort of modeling problem, the best fit chosen from among a class of models need not necessarily be a *good* fit to the data if the model class itself does not contain a sufficiently rich set of models from which to choose. The concept of model *goodness-of-fit* is therefore important. But, while this concept is fairly well developed in standard modeling contexts, such as linear modeling, it is arguably still in its infancy as far as network graph modeling is concerned.

For ERGMs, the current practice in assessing goodness-of-fit is to first simulate numerous random graphs from the fitted model and then compare high-level characteristics of these graphs with those of the originally observed graph. Examples of such characteristics include the distribution of any number of the various summaries of network structure encountered in Chap. 4, such as degree, centrality, and geodesic distance. If the characteristics of the observed network graph are too poor of a match to the typical values arising from realizations of the fitted random graph model, then

this suggests systematic differences between the specified class of models and the data, and therefore a lack of goodness-of-fit.¹¹

To assess the goodness-of-fit of our model in (6.13), as fit by `ergm`, the function `gof` in **ergm** runs the necessary Monte Carlo simulation and calculates comparisons with the original network graph in terms of the distribution of degree, geodesic length, and edge-wise shared partners (i.e., the number of neighbors shared by a pair of vertices defining an edge).

```
#6.11 1 > gof.lazega.ergm <- gof(lazega.ergm.fit)
```

The results of these computations may then be plotted,

```
#6.12 1 > par(mfrow=c(1, 3))
      2 > plot(gof.lazega.ergm)
```

as shown in Fig. 6.1. They indicate that—on these particular characteristics—the fit of the model is quite good overall.

6.3 Network Block Models

We have seen that the structure of an ERGM closely parallels that of a standard regression model in statistics. The presence or absence of network edges (i.e., the Y_{ij}) is taken to be the response variable, while the role of the predictor variables is played by some combination of network summary statistics (i.e., endogenous variables) and functions of vertex and edge attributes (i.e., incorporating exogenous effects). In this section, we examine the class of network *block models*, which are instead analogous to classical mixture models.¹²

Recall that, in our analysis of the network of lawyer collaborations in the previous section, we used as predictors the sums of indicators that various attributes (e.g., practice or gender) were shared between vertex pairs. Importantly, while this choice may seem sensible from a practical perspective, it also reflects the potential impact on the formation of network ties of a key principle in social network theory—that of structural equivalence, i.e., the similarity of network positions and social roles. See [144, Chap. 9], for example. In general, we may think of vertices in a network as belonging to classes, and the propensity to establish ties between vertex pairs as depending on the class membership of the two vertices. With network block models these concepts are made precise.

¹¹ Goodness-of-fit has been found to be particularly important where ERGMs are concerned, due in large part to the issue of potential model degeneracy.

¹² A random variable X is said to follow a Q -class mixture distribution if its probability density function is of the form $f(x) = \sum_{q=1}^Q \alpha_q f_q(x)$, for class-specific densities f_q , where the mixing weights α_q are all non-negative and sum to one.

Goodness-of-fit diagnostics

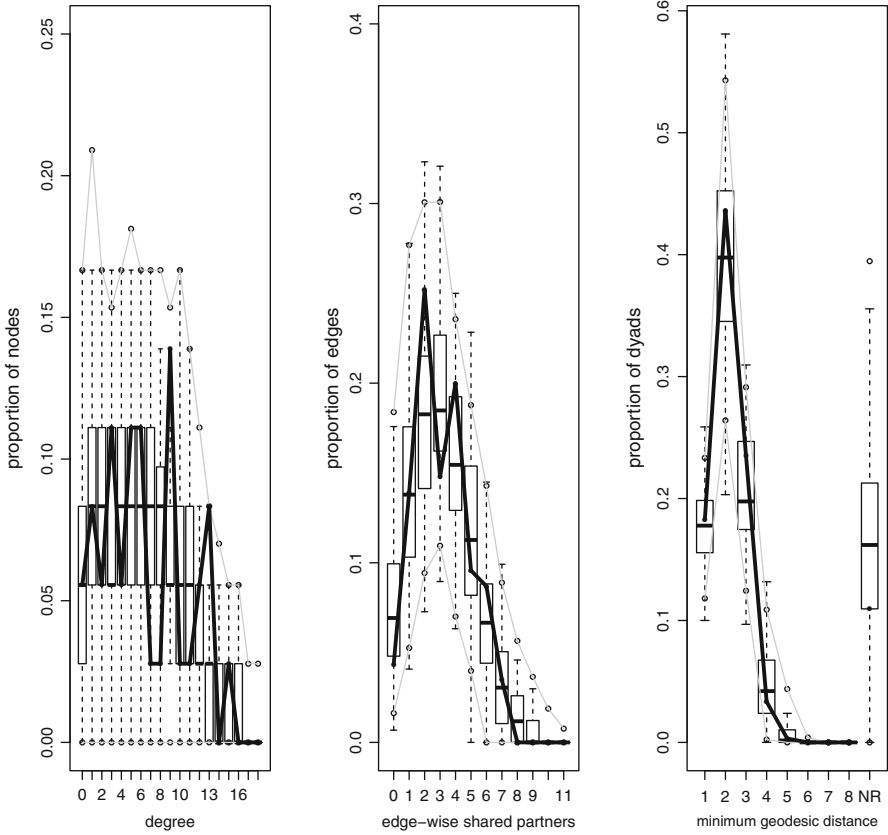


Fig. 6.1 Goodness-of-fit plots comparing original Lazega lawyer network and Monte Carlo realizations from the model in (6.13), with the parameters obtained by `ergm`. Comparisons are made based on the distribution of degree, edge-wise shared partners, and geodesic distance, represented by *box-plots* and *curves* showing 10th and 90th quantiles. Values for the Lazega network itself are shown with **bold solid lines**. In the distribution of geodesic distances between pairs, the rightmost box-plot is separate and corresponds to the proportion of nonreachable pairs

6.3.1 Model Specification

Suppose that each vertex $i \in V$ of a graph $G = (V, E)$ can belong to one of Q classes, say $\mathcal{C}_1, \dots, \mathcal{C}_Q$. And furthermore, suppose that we know the class label $q = q(i)$ for each vertex i . A *block model* for G specifies that each element Y_{ij} of the adjacency matrix $\mathbf{Y}$ is, conditional on the class labels q and r of vertices i and j , respectively, an independent Bernoulli random variable, with probability π_{qr} . For an undirected graph, $\pi_{qr} = \pi_{rq}$.

The block model is hence a variant of the Bernoulli random graph model, where the probabilities of an edge are restricted to be one of only Q^2 possible values π_{qr} .

Furthermore, in analogy to (6.5), this model can be represented in the form of an ERGM, i.e.,

$$\mathbb{P}_{\theta}(\mathbf{Y} = \mathbf{y}) = \left(\frac{1}{\kappa}\right) \exp \left\{ \sum_{q,r} \theta_{qr} L_{qr}(\mathbf{y}) \right\}, \quad (6.17)$$

where $L_{qr}(\mathbf{y})$ is the number of edges in the observed graph $\mathbf{y}$ connecting pairs of vertices of classes q and r .

Nevertheless, the assumption that the class membership of vertices is known or, moreover, that the ‘true’ classes $\mathcal{C}_1, \dots, \mathcal{C}_Q$ have been correctly specified, is generally considered untenable in practice. More common, therefore, is the use of a *stochastic block model* (SBM) [121]. This model specifies only that there are Q classes, for some Q , but does not specify the nature of those classes nor the class membership of the individual vertices. Rather, it dictates simply that the class membership of each vertex i be determined independently, according to a common distribution on the set $\{1, \dots, Q\}$.

Formally, let $Z_{iq} = 1$ if vertex i is of class q , and zero otherwise. Under a stochastic block model, the vectors $\mathbf{Z}_i = (Z_{i1}, \dots, Z_{iQ})$ are determined independently, where $\mathbb{P}(Z_{iq} = 1) = \alpha_q$ and $\sum_{q=1}^Q \alpha_q = 1$. Then, conditional on the values $\{\mathbf{Z}_i\}$, the entries Y_{ij} are again modeled as independent Bernoulli random variables, with probabilities π_{qr} , as in the non-stochastic block model.

A stochastic block model is thus, effectively, a mixture of classical random graph models. As such, many of the properties of the random graphs G resulting from this model may be worked out in terms of the underlying model parameters. See [41], for example, who refer to this class of models as a ‘mixture model for random graphs’.

Various extensions of the stochastic block model have in turn been proposed, although we will not pursue them here. For example, the class of mixed-membership stochastic block models allows vertices to be members of more than one class [2]. Similarly, the class of degree-corrected stochastic block models aims to produce mixtures of random graphs that have more heterogeneous degree distributions than the Poisson distribution corresponding to the classical random graph (e.g., [34, 86]).

6.3.2 Model Fitting

A non-stochastic block model can be fit in a straightforward fashion. The only parameters to be estimated are the edge probabilities π_{qr} , and the maximum likelihood estimates—which are natural here—are simply the corresponding empirical frequencies.

In the case of stochastic block models, both the (now conditional) edge probabilities π_{qr} and the class membership probabilities α_q must be estimated. While this may not seem like much of a change over the ordinary block model, the task of model fitting becomes decidedly more complex in this setting. In order to see why, note that the log-likelihood for the joint distribution of the adjacency matrix $\mathbf{Y}$ and the class membership vectors $\{\mathbf{Z}_i\}$, i.e., the complete-data log-likelihood, is of the form

$$\ell(\mathbf{y}; \{\mathbf{z}_i\}) = \sum_i \sum_q z_{iq} \log \alpha_q + \frac{1}{2} \sum_{i \neq j} \sum_{q \neq r} z_{iq} z_{jr} \log b(y_{ij}; \pi_{qr}) , \quad (6.18)$$

where $b(y; \pi) = \pi^y (1 - \pi)^{1-y}$. In principle, the likelihood of the observed data is obtained then by summing the complete-data likelihood over all possible values of $\{\mathbf{z}_i\}$. Unfortunately, to do so typically is intractable in problems of any real interest. As a result, computationally intensive methods must be used to produce estimates based on this likelihood.

The expectation-maximization (EM) algorithm [109] is a natural choice here. Effectively, given a current estimate of the π_{qr} , expected values of the $\mathbf{Z}_i$ are computed, conditional on $\mathbf{Y} = \mathbf{y}$. These values in turn are used to compute new estimates of the π_{qr} , using (conditional) maximum likelihood principles. The two steps are repeated in an alternating fashion, until convergence. But the first (i.e., expectation) step cannot be done in closed form, which greatly reduces the appeal of the algorithm.

Instead, a number of methods that approximate or alter the original maximum likelihood problem have been proposed in the literature. The R package **mixer** implements a number of these, with the default method being a so-called variational approach, which optimizes a lower bound on the likelihood of the observed data.

To illustrate, we use the network `fblog` of French political blogs introduced in Chap. 3.5. Recall that each blog is annotated as being associated with one of nine French political parties. Of course, these annotations do not necessarily correspond to an actual ‘true’ set of class groupings for these blogs, in the sense intended by the relatively simple form of the stochastic block model. Nevertheless, the context of the data (i.e., political blogs in the run-up to the French 2007 presidential election), as well as the various visualizations of this network in Chap. 3.5, suggest that it is likely the stochastic block model is not an unreasonable approximation to reality in this case.

Using the function `mixer` in **mixer**, a fit to the observed network graph $\mathbf{y}$ is obtained through

```
#6.13 1 > library(mixer)
      2 > setSeed(42)
      3 > fblog.sbm <- mixer(as.matrix(get.adjacency(fblog)),
      4 +                    qmin=2, qmax=15)
```

Note that we have specified only that the total number of classes Q be between 2 and 15. The so-called integration classification likelihood (ICL) criterion is used by `mixer` to select the number of classes fit to the network. This criterion is similar in spirit to various information criteria popular in standard regression modeling (e.g., Akaike’s information (AIC), Bayesian information (BIC), etc.), but adapted specifically to clustering problems.

Examining the model output

```
#6.14 1 > fblog.sbm.output <- getModel(fblog.sbm)
      2 > names(fblog.sbm.output)
      3 [1] "q"          "criterion" "alphas"    "Pis"
      4 [5] "Taus"
```

we see that the network of French blogs has been fit with

```
#6.15 1 > fblog.sbm.output$q
      2 [1] 12
```

classes, in estimated proportions

```
#6.16 1 > fblog.sbm.output$alphas
      2 [1] 0.15294139 0.13007188 0.12307831 0.05729167
      3 [5] 0.13581585 0.03123927 0.09967103 0.09795210
      4 [9] 0.01041667 0.02088946 0.12500738 0.01562500
```

The output from a fitted stochastic block model also allows for the assignment of vertices to classes. Thus stochastic block models may be used as a model-based method of graph partitioning, complementing the other methods introduced in Chap. 4.4. Specifically, as mentioned above in the sketch of the EM algorithm, in producing estimates of the parameters π_{qr} and α_q , algorithms of this type (i.e., including the variational approximation used by `mixer`) necessarily calculate estimates of the expected values of the $\mathbf{Z}_i$, conditional on $\mathbf{Y} = \mathbf{y}$. That is, they calculate estimates of the posterior probability of class membership, which may then be used to determine class assignments.

For example, examining the estimates for the first three vertices in the French blog network

```
#6.17 1 > fblog.sbm.output$Taus[, 1:3]
      2           [,1]           [,2]           [,3]
      3 [1,] 9.999820e-01 9.162358e-04 9.999910e-01
      4 [2,] 1.182601e-05 1.000000e-10 5.169635e-07
      5 [3,] 4.702876e-06 9.990596e-01 8.427162e-06
      6 [4,] 1.000000e-10 1.000000e-10 1.000000e-10
      7 [5,] 1.094414e-06 1.000000e-10 5.707788e-09
      8 [6,] 1.000000e-10 1.000000e-10 1.000000e-10
      9 [7,] 3.451962e-07 2.418009e-05 4.619964e-08
     10 [8,] 1.000000e-10 1.000000e-10 1.000000e-10
     11 [9,] 1.000000e-10 1.000000e-10 1.000000e-10
     12 [10,] 1.000000e-10 1.000000e-10 1.000000e-10
     13 [11,] 4.531089e-09 1.000000e-10 1.000000e-10
     14 [12,] 1.000000e-10 1.000000e-10 1.000000e-10
```

we see that an assignment rule based on the *maximum a posteriori* criterion would place the first and third vertices in class 1, and the second, in class 3.

Interestingly, the posterior probability vectors for these three vertices concentrate their weight quite strongly on the most probable class. This fact is reflected in the entropy¹³ values of these vectors

```
#6.18 1 > my.ent <- function(x) { -sum(x*log(x, 2)) }
      2 > apply(fblog.sbm.output$Taus[, 1:3], 2, my.ent)
      3 [1] 0.0003319527 0.0109735939 0.0001671334
```

¹³ The entropy of a discrete probability distribution $\mathbf{p} = (p_1, \dots, p_Q)$ is defined as $H(\mathbf{p}) = -\sum_{q=1}^Q p_q \log_2 p_q$, with smaller values indicating a distribution concentrated on fewer classes. This value is bounded above by $\log_2 Q$, corresponding to a uniform distribution on $\{1, \dots, Q\}$.

which are quite small compared to the value

```
#6.19 1 > log(fbblog.sbm.output$q, 2)
      2 [1] 3.584963
```

corresponding to the extreme case of a uniform distribution across the 12 classes. The same observation seems to hold for the vast majority of the vertices

```
#6.20 1 > summary(apply(fbblog.sbm.output$Taus, 2, my.ent))
      2      Min.      1st Qu.      Median      Mean      3rd Qu.
      3 0.0000000 0.0000000 0.0000003 0.0343200 0.0006172
      4      Max.
      5 1.0100000
```

6.3.3 Goodness-of-Fit

In assessing the goodness-of-fit of a stochastic block model we could, of course, use the same types of simulation-based methods we employed in the analysis of ERGMs (i.e., illustrated in Fig. 6.1). However, the particular form of a stochastic block model lends itself as well to certain other more model-specific devices. A selection of summaries produced by the **mixer** package are displayed in Fig. 6.2.

```
#6.21 1 > plot(fbblog.sbm, classes=as.factor(V(fbblog)$PolParty))
```

We see, for example, that while the fitted model has $Q = 12$ classes, the integrated conditional likelihood (ICL) criteria seems to suggest there is some latitude in this choice, with anywhere from 8 to 12 classes being reasonable. Examination of the adjacency matrix **y**, with rows and columns reorganized by the assigned vertex classes, indicates that there are seven larger classes, and five smaller classes. Furthermore, while it appears that the vertices in some of these classes are primarily connected with other vertices within their respective classes, among those other classes in which vertices show a propensity towards inter-class connections there seems to be, in most cases, a tendency towards connecting selectively with vertices of only certain other classes.

With respect to the degree distribution, it appears that the distribution corresponding to the fitted stochastic block model (shown as a blue curve) is able to describe the observed degree distribution (shown as a yellow histogram) reasonably well, although the body of the fitted distribution is arguably shifted to the right somewhat of the observed distribution.

Finally, it is of interest to consider to what extent the graph partitioning induced by the vertex class assignments matches the grouping of these blogs according to their political party status. This comparison is summarized in the last plot in Fig. 6.2. Here the circles, corresponding to the 12 vertex classes, and proportional in size to the number of blogs assigned to each class, are further broken down according to the relative proportion of political parties to which the blogs correspond, displayed in the form of pie charts. Connecting the circles are edges drawn with a thickness in proportion to the estimated probability that blogs in the two respective groups link

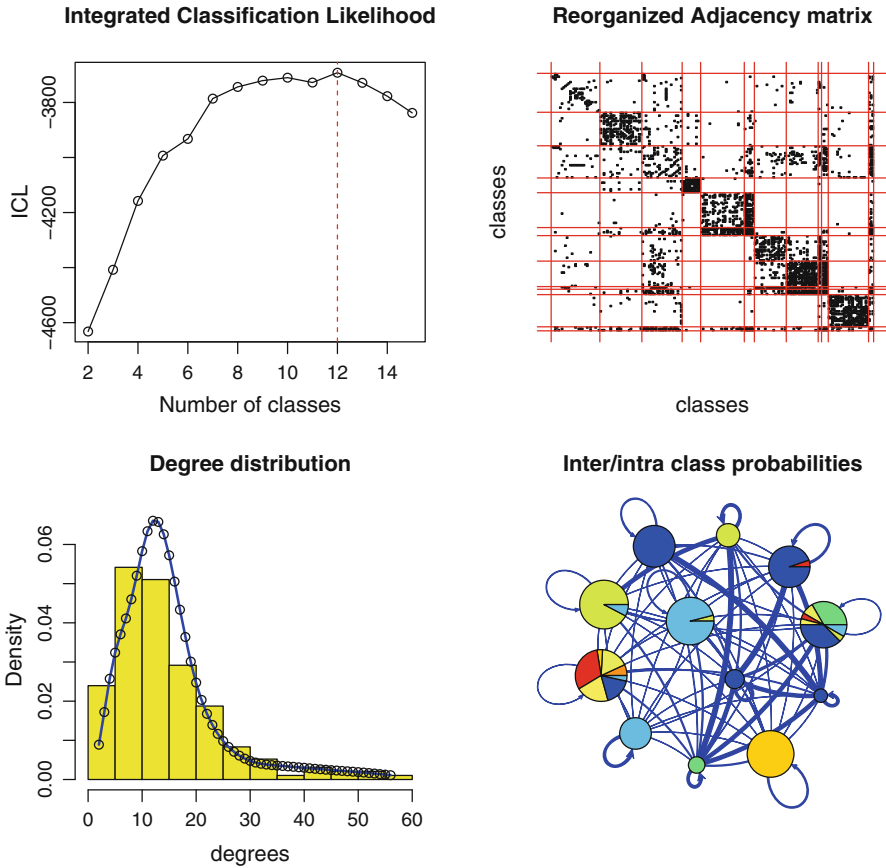


Fig. 6.2 Various plots summarizing the goodness-of-fit for the stochastic block model analysis of the French political blog network

to each other (i.e., in proportion to the estimated π_{qr}). Note that this plot may be contrasted with the coarse-level visualization of the original French blog network in Fig. 3.7.

A close examination of the pie charts yields, for example, that while the blogs in most of the 12 classes are quite homogeneous in their political party affiliations, two of the larger classes have a rather heterogeneous set of affiliations represented. In addition, two of the political parties (shown in light blue and light green) appear to be split largely between two classes, one larger and one smaller, while another (blue) appears to be mainly split among four classes, two larger and two smaller. This latter observation might suggest that the model has chosen to use too many classes. Alternatively, it could instead indicate that there is actually splintering within these political parties.

6.4 Latent Network Models

From the perspective of statistical modeling, one key innovation underlying stochastic block models and their extensions is the incorporation of latent variables, in the form of vertex classes. That is, the use of variables that are unobserved but which play a role in determining the probability that vertex pairs are incident to each other. The principle of latent variables, common in many other areas of statistical modeling, has been adopted in a quite general sense with the class of *latent network models*. We draw on Hoff [73] in our development of these models below, and illustrate their usage with the R package **eigenmodel**, by the same author.

6.4.1 General Formulation

The incorporation of latent variables in network models for a random graph $G = (V, E)$ can be motivated by results of Hoover [77] and Aldous [4]. Specifically, in the absence of any covariate information, the assumption of exchangeability¹⁴ of the vertices $v \in V$ is natural, and from this an argument can be made that each element Y_{ij} of the adjacency matrix $\mathbf{Y}$ can be expressed in the form

$$Y_{ij} = h(\infty, u_i, u_j, \varepsilon_{ij}) \quad , \quad (6.19)$$

where ∞ is a constant, the u_i are independent and identically distributed latent variables, the ε_{ij} are independent and identically distributed pair-specific effects, and the function h is symmetric in its second and third arguments. In other words, under exchangeability, any random adjacency matrix $\mathbf{Y}$ can be written as a function of latent variables.

Given the generality of the expression in (6.19), there are clearly many possible latent network models we might formulate. If we specify that (i) the ε_{ij} are distributed as standard normal random variables, (ii) the latent variables u_i, u_j enter into h only through a symmetric function $\alpha(u_i, u_j)$, and (iii) the function h is simply an indicator as to whether or not (i.e., one or zero) its argument is positive, and if in addition we augment the parameter ∞ to include a linear combination of pair-specific covariates, i.e., $\mathbf{x}_{ij}^T \beta$, then we arrive at a network version of a so-called probit model. Under this model, the Y_{ij} are conditionally independent, with distributions

$$\mathbb{P}(Y_{ij} = 1 \mid \mathbf{X}_{ij} = \mathbf{x}_{ij}) = \Phi \left(\infty + \mathbf{x}_{ij}^T \beta + \alpha(u_i, u_j) \right) \quad , \quad (6.20)$$

where Φ is the cumulative distribution function of a standard normal random variable.¹⁵

¹⁴ A set of random variables is said to be *exchangeable* if their joint distribution is the same for any ordering.

¹⁵ In general, a probit model specifies, for a binary response Y , as a function of covariates $\mathbf{x}$, that $\mathbb{P}(Y = 1 \mid \mathbf{X} = \mathbf{x}) = \Phi(\mathbf{x}^T \beta)$, for some β .

If we denote the probabilities in (6.20) as p_{ij} , then the conditional model for $\mathbf{Y}$ as a whole takes the form

$$\mathbb{P}(\mathbf{Y} = \mathbf{y} | \mathbf{X}, u_1, \dots, u_{N_v}) = \prod_{i < j} p_{ij}^{y_{ij}} (1 - p_{ij})^{1 - y_{ij}}. \quad (6.21)$$

That is, conditional on the covariates $\mathbf{X}$ and the latent variables $u_1, \dots, u_{N_v}$, this model for G has the form of a Bernoulli random graph model, with probabilities p_{ij} specific to each vertex pair i, j . Note that complete specification of the full model requires that a choice of distribution be made for the latent variables as well. We will revisit this point later, in Sect. 6.4.3, after first exploring the issue of selecting the form of the function $\alpha(\cdot, \cdot)$.

6.4.2 Specifying the Latent Effects

The effect of the latent variables u on the probability of there being an edge between vertex pairs is largely dictated by the form of the function $\alpha(\cdot, \cdot)$. There have been a number of options explored in the literature to date. We remark briefly on three such options here.

A latent class model—analogue to the stochastic block models of Sect. 6.3—can be formulated by specifying that the u_i take values in the set $\{1, \dots, Q\}$, and that $\alpha(u_i, u_j) = m_{u_i, u_j}$, for a symmetric matrix $\mathbf{M} = [m_{q,r}]$ of real-valued entries $m_{q,r}$. As remarked previously, the use of latent classes encodes into the model a notion of the principle of structural equivalence from social network theory.

Alternatively, the principle of homophily (i.e., the tendency of similar individuals to associate with each other) suggests an alternative choice, based on the concept of distance in a latent space. In this formulation, the latent variables u_i are simply vectors $(u_{i1}, \dots, u_{iQ})^T$, of real numbers, interpreted as important but unknown characteristics of vertices that influence whether each establishes edges (e.g., social ties) with the others, and—importantly—vertices with more similar characteristics are expected to be more likely to establish an edge. Accordingly, the latent effects are specified as $\alpha(u_i, u_j) = -|u_i - u_j|$, for some distance metric $|\cdot|$, and the models are known as latent distance models.

Hoff [73] has suggested a third approach to specifying latent effects that combines the two approaches above, based on principles of eigen-analysis. Here the u_i are again Q -length random vectors, but the latent effects are given the form $\alpha(u_i, u_j) = u_i^T \Lambda u_j$, where Λ is a $Q \times Q$ diagonal matrix. Recall that the latent variables u are modeled as independent and identically distributed random vectors from the same distribution, and hence the correlation between each pair u_i and u_j is zero. While this is not the same as linear independence, in a linear algebraic sense, nevertheless it may be interpreted as saying that the u_i will be orthogonal ‘in expectation’. Gathering the u_i into a matrix $\mathbf{U} = [u_1, \dots, u_Q]$, the product $\mathbf{U} \Lambda \mathbf{U}^T$ therefore may be thought of as being in the spirit of an eigen-decomposition of the matrix of all pairwise latent effects $\alpha(u_i, u_j)$. Hoff refers to this model as an ‘eigenmodel’.

The collection of eigenmodels can be shown to include the collection of latent class models in a formal sense, in that the set of matrices of latent effects that can be generated by the latter model is contained within that of the former model. In addition, there is a similar (albeit weaker) relationship between the collection of eigenmodels and the collection of latent distance models. As a result, the eigenmodel can be said to generalize both of these classes and, hence, its use allows for models that incorporate a blending of the principles of both structural equivalence and homophily. The manner in which the two principles are to be blended can be determined in a data-driven manner, through the process of model fitting.

6.4.3 Model Fitting

By construction, the latent network model has a hierarchical specification, so a Bayesian approach to inference is natural here. The package **eigenmodel** implements the eigenmodel formulation described above and will be the one with which we illustrate here.¹⁶

The function `eigenmodel.mcmc` in **eigenmodel** uses Monte Carlo Markov Chain (MCMC) techniques to simulate from the relevant posterior distributions, using largely conjugate priors to complete the model specification. Of particular interest are the parameter β (describing the effects of pair-specific covariates $\mathbf{x}_{ij}$), the elements of the diagonal matrix Λ (summarizing the relative importance of each latent vector u_i), and the latent vectors themselves. Since the inferred latent vectors $\hat{u}_i$ are not orthogonal, it is useful in interpreting model output to use in their place the eigenvectors of the matrix $\hat{\mathbf{U}}\hat{\Lambda}\hat{\mathbf{U}}^T$.

The network `lazega` of collaborations among lawyers allows for demonstration of a number of the concepts we have discussed so far. Recall that this network involved 36 lawyers, at three different office locations, involved in two types of practice (i.e., corporate and litigation).

```
#6.22 1 > summary(lazega)
      2 IGRAPH UN-- 36 115 --
      3 attr: name (v/c), Seniority (v/n), Status (v/n),
      4   Gender (v/n), Office (v/n), Years (v/n), Age
      5   (v/n), Practice (v/n), School (v/n)
```

We might hypothesize that collaboration in this setting is driven, at least in part, by similarity of practice, a form of homophily. On the other hand, we could similarly hypothesize that collaboration is instead driven by shared office location, which could be interpreted as a proxy for distance. Because the eigenmodel formulation of latent network models is able to capture aspects of both distance and homophily, it is interesting to compare the fitted models that we obtain for three different

¹⁶ The package **latentnet**, in the **statnet** suite of tools, implements other variants of latent network models, such as latent distance models.

eigenmodels, specifying (i) no pair-specific covariates, (ii) a covariate for common practice, and (iii) a covariate for shared office location, respectively.

To fit the model with no pair-specific covariates and a latent eigen-space of $Q = 2$ dimensions is accomplished as follows.¹⁷

```
#6.23 1 > library(eigenmodel)
      2 > set.seed(42)
      3 > A <- get.adjacency(lazega, sparse=FALSE)
      4 > lazega.leig.fit1 <- eigenmodel_mcmc(A, R=2, S=11000,
      5 +   burn=10000)
```

In order to include the effects of common practice, we create an array with that information

```
#6.24 1 > same.prac.op <- v.attr.lazega$Practice %o%
      2 +   v.attr.lazega$Practice
      3 > same.prac <- matrix(as.numeric(same.prac.op
      4 +   %in% c(1, 4, 9)), 36, 36)
      5 > same.prac <- array(same.prac,dim=c(36, 36, 1))
```

and fit the model with this additional argument

```
#6.25 1 > lazega.leig.fit2 <- eigenmodel_mcmc(A, same.prac, R=2,
      2 +   S=11000, burn=10000)
```

Finally, we do similarly for the model that includes information on shared office locations.

```
#6.26 1 > same.off.op <- v.attr.lazega$Office %o%
      2 +   v.attr.lazega$Office
      3 > same.off <- matrix(as.numeric(same.off.op %in%
      4 +   c(1, 4, 9)), 36, 36)
      5 > same.off <- array(same.off,dim=c(36, 36, 1))
      6 > lazega.leig.fit3 <- eigenmodel_mcmc(A, same.off,
      7 +   R=2, S=11000, burn=10000)
```

In order to compare the representation of the network lazega in each of the underlying two-dimensional latent spaces inferred for these models, we extract the eigenvectors for each fitted model

```
#6.27 1 > lat.sp.1 <-
      2 +   eigen(lazega.leig.fit1$ULU_postmean)$vec[, 1:2]
      3 > lat.sp.2 <-
      4 +   eigen(lazega.leig.fit2$ULU_postmean)$vec[, 1:2]
      5 > lat.sp.3 <-
      6 +   eigen(lazega.leig.fit3$ULU_postmean)$vec[, 1:2]
```

and plot the network in **igraph** using these coordinates as the layout.¹⁸ For example,

¹⁷ The arguments *S* and *burn* chosen in our example ask that a ‘burn-in’ of 10,000 iterations be used to initiate our MCMC sampler, after which the following 1,000 iterations are used to perform posterior inference.

¹⁸ Conventions of vertex color, shape, and label are the same as in Fig. 1.1 in Chap. 1, and are specified in R in the same manner as seen in Chap. 3.

```
#6.28 1 > colbar <- c("red", "dodgerblue", "goldenrod")
      2 > v.colors <- colbar[V(lazega)$Office]
      3 > v.shapes <- c("circle", "square")[V(lazega)$Practice]
      4 > v.size <- 3.5*sqrt(V(lazega)$Years)
      5 > v.label <- V(lazega)$Seniority
      6 > plot(lazega, layout=lat.sp.1, vertex.color=v.colors,
      7 +   vertex.shape=v.shapes, vertex.size=v.size,
      8 +   vertex.label=v.label)
```

generates the visualization corresponding to the fit without any pair-specific covariates, and those for the other two models are obtained similarly (Fig. 6.3).

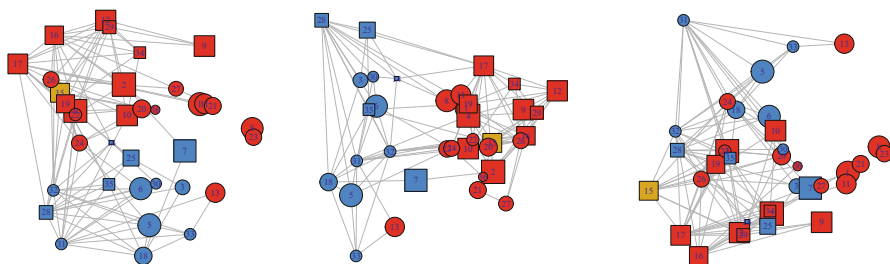


Fig. 6.3 Visualizations of the network of Lazega's lawyers, with layouts determined according to the inferred latent eigenvectors in models with no pair-specific covariates (*left*), a covariate for common practice (*center*), and a covariate for shared office location (*right*)

Examination of these three visualizations indicates that while the first two are somewhat similar, the third is distinct. In particular, while the lawyers in the first two visualizations appear to be clustered into two main groups distinguished largely by common office location (i.e., color), in the third there appears to be only one main cluster. These observations suggest that common practice explains comparatively much less coarse-scale network structure than shared office location. And, indeed, when shared office location is taken into account, there is decidedly less structure left to be captured by the latent variables. Comparison of the posterior means of the elements in $\hat{\Lambda}$ for each of these models reinforces these conclusions, in that for the first two models there is one eigenvalue that clearly dominates the other, corresponding to the axis on which we obtain a clear separation between the two groups, whereas for the third model the eigenvalues are comparable in their magnitude.

```
#6.29 1 > apply(lazega.leig.fit1$L_postsamp, 2, mean)
      2 [1] 0.2603655 1.0384032
      3 > apply(lazega.leig.fit2$L_postsamp, 2, mean)
      4 [1] 0.9083401 -0.1385321
      5 > apply(lazega.leig.fit3$L_postsamp, 2, mean)
      6 [1] 0.5970403 0.3112896
```

6.4.4 Goodness-of-Fit

Here again, in assessing the goodness-of-fit of a latent network model we could, of course, use the same types of simulation-based methods we employed in the analysis of ERGMs (i.e., illustrated in Fig. 6.1). Alternatively, a more global sense of goodness-of-fit can be obtained by using principles of cross-validation. Specifically, a common practice in network modeling is to assess the accuracy with which, in fitting a model to a certain subset of the network, the remaining part of the network may be predicted. This notion usually is implemented through K -fold cross-validation, wherein the observed values y_{ij} are partitioned into K subsets (e.g., $K = 5$ is a standard choice), and the values in those subsets are predicted after training the same model on each of the complements of those subsets.

For example, consider the model fit above to the data `lazega` with no pair-specific covariates. After initiating a permutation of the $36 \times 35/2 = 630$ unique off-diagonal elements of the symmetric adjacency matrix, and initializing vector-based representations of the corresponding lower triangular portion of this matrix

```
#6.30 1 > perm.index <- sample(1:630)
      2 > nfolds <- 5
      3 > nmiss <- 630/nfolds
      4 > Avec <- A[lower.tri(A)]
      5 > Avec.pred1 <- numeric(length(Avec))
```

the process of cross-validation is implemented in the following lines.

```
#6.31 1 > for(i in seq(1, nfolds)){
      2 >   # Index of missing values.
      3 >   miss.index <- seq(((i-1) * nmiss + 1),
      4 +     (i * nmiss), 1)
      5 >   A.miss.index <- perm.index[miss.index]
      6 >
      7 >   # Fill a new Atemp appropriately with NA's.
      8 >   Avec.temp <- Avec
      9 >   Avec.temp[A.miss.index] <-
    10 +     rep("NA", length(A.miss.index))
    11 >   Avec.temp <- as.numeric(Avec.temp)
    12 >   Atemp <- matrix(0, 36, 36)
    13 >   Atemp[lower.tri(Atemp)] <- Avec.temp
    14 >   Atemp <- Atemp + t(Atemp)
    15 >
    16 >   # Now fit model and predict.
    17 >   Y <- Atemp
    18 >
    19 >   modell1.fit <- eigenmodel_mcmc(Y, R=2,
    20 +     S=11000, burn=10000)
    21 >   modell1.pred <- modell1.fit$Y_postmean
    22 >   modell1.pred.vec <-
    23 +     modell1.pred[lower.tri(modell1.pred)]
    24 >   Avec.pred1[A.miss.index] <-
    25 +     modell1.pred.vec[A.miss.index]
    26 > }
```

Similarly, we can do the same for the models fit above with pair-specific covariates for common practice and shared office location, respectively, yielding, say, `Avec.pred2` and `Avec.pred3`. The results of the predictions generated under each of these three models can be assessed by examination of the corresponding receiver operating characteristic (ROC) curves.¹⁹

For example, using the package **ROCR**, an ROC curve for the predictions based on our first model are generated as follows.

```
#6.32 1 > library(ROCR)
      2 > pred1 <- prediction(Avec.pred1, Avec)
      3 > perf1 <- performance(pred1, "tpr", "fpr")
      4 > plot(perf1, col="blue", lwd=3)
```

The ROC curves for each of our three latent network models for the Lazega lawyer network are shown in Fig. 6.4. We see that from the perspective of predicting edge

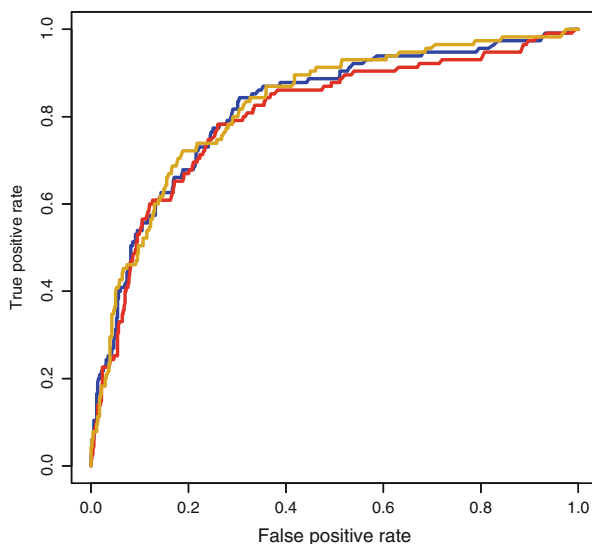


Fig. 6.4 ROC curves comparing the goodness-of-fit to the Lazega network of lawyer collaboration for three different eigenmodels, specifying (i) no pair-specific covariates (*blue*), (ii) a covariate for common practice (*red*), and (iii) a covariate for shared office location (*yellow*), respectively

status, all three models appear to be comparable in their performance and to perform reasonably well, with an area under the curve (AUC) of roughly 80 %.

¹⁹ An ROC curve is used commonly in classification problems. The term refers to a curve obtained by plotting the true positive rate of a classifier against the true negative rate, as a threshold (or similar parameter) is varied across its natural range, where the threshold is applied to the predicted values to discriminate between two classes of interest. Here, since the predictions are posterior probabilities, the threshold is varied from 0 to 1, with vertex pairs for which the posterior probability of an edge is above threshold being predicted to have an edge.


```
#6.33 1 > perf1.auc <- performance(pred1, "auc")
      2 > slot(perf1.auc, "y.values")
      3 [[1]]
      4 [1] 0.820515
```

6.5 Additional Reading

Of the three model classes discussed in this chapter, ERGMs have by far the longest and most extensive development, which has been summarized in the review article by Robins et al. [[126](#)] and detailed in the book by Lusher et al. [[105](#)]. For network block models and latent network models, the seminal articles (such as those referenced above) are at this time still the best resources for additional details.